Python Decorators: A Beginner-Friendly Deep Dive

If you have ever seen a line like `@something` sitting on top of a Python function and wondered what that little `@` symbol does, this article is for you. Decorators are one of Python's most elegant features. They let you add behavior to a function — logging, timing, access control, caching — without ever touching the function's own code.

At first glance decorators can look like magic syntax. The goal here is to dissolve that magic completely. We will start from a few simple facts about how Python treats functions, build a decorator by hand from those facts, and only then introduce the `@` shortcut. By the end you will not just *use* decorators; you will understand exactly what they are doing and why. Along the way we will solve real problems you are likely to meet in actual projects.

Part 1: The Foundation — Functions Are Objects

Before decorators make any sense, you need one key idea firmly in place: **in Python, functions are objects, just like numbers, strings, and lists.** This means you can do things with functions that might surprise you. Three abilities matter most.

Functions can be assigned to variables

A function name is just a label pointing at a function object. You can attach more labels to the same object:

```python
def greet(name):
    return f"Hello, {name}"

say = greet            # 'say' now points to the same function
print(say("Alice"))    # Hello, Alice
```

Notice there are no parentheses after `greet` on the assignment line. `greet` refers to the function itself; `greet()` *calls* it. That distinction is the single most important thing to keep straight in this entire topic.

Functions can be passed as arguments

Because a function is just an object, you can hand it to another function:

```python
```

```python
def shout(text):
    return text.upper()

def apply(func, value):
    return func(value)     # call whatever function we were given

print(apply(shout, "hello"))   # HELLO
```

A function that takes another function as input (or returns one) is called a **higher-order function**. Decorators are a special, polished kind of higher-order function.

Functions can be returned from functions

You can also define a function *inside* another function and return it:

python

```python
def make_multiplier(factor):
    def multiplier(n):
        return n * factor      # 'factor' is remembered from the outer function
    return multiplier          # return the inner function (no parentheses!)

times3 = make_multiplier(3)
print(times3(10))              # 30
```

There is something subtle and powerful happening here. The inner `multiplier` function uses `factor`, a variable from the outer function. Even after `make_multiplier` has finished running and returned, the returned function *still remembers* the value of `factor`. This memory of the surrounding variables is called a **closure**, and it is the engine that makes decorators work. Hold onto this idea — we will use it constantly.

Part 2: Building a Decorator by Hand

Now we can combine those three abilities. A **decorator** is simply a function that takes another function, wraps it in some extra behavior, and returns the wrapped version.

Let's build one step by step. Suppose we want to print a message before and after a function runs:

python

```python
def my_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()                          # call the original function
        print("After the function runs")
    return wrapper                      # hand back the wrapped version

def say_hi():
    print("Hi")

say_hi = my_decorator(say_hi)           # wrap it, reassign the name
say_hi()
```

Output:

```
Before the function runs
Hi
After the function runs
```

Walk through exactly what happened, because this is the whole concept in miniature:

1. `my_decorator` receives the original `say_hi` function as `func`.
2. Inside, it defines a brand-new function `wrapper` that adds the two `print` statements *around* a call to `func()`.
3. It returns `wrapper`. Thanks to closures, `wrapper` permanently remembers which `func` it is supposed to call.
4. We reassign the name `say_hi` to point at this new wrapper. Now whenever anyone calls `say_hi()`, they actually run the wrapper, which sandwiches the original behavior between the two messages.

The original `say_hi` function was never modified. We simply wrapped a new layer around it. That is the entire essence of decoration.

Part 3: The `@` Syntax

The line `say_hi = my_decorator(say_hi)` is so common that Python gives it a dedicated shortcut: place `@decorator_name` on the line directly above the function definition. These two snippets are **exactly equivalent**:

python

```
# The manual way
def say_hi():
    print("Hi")
say_hi = my_decorator(say_hi)

# The @ way — identical in effect
@my_decorator
def say_hi():
    print("Hi")
```

So `@my_decorator` above a function definition is pure syntactic sugar meaning "take this function, pass it through `my_decorator`, and rebind the name to the result." Nothing more mysterious than that. Here it is in full:

python

```
def announce(func):
    def wrapper():
        print("Starting")
        func()
        print("Done")
    return wrapper

@announce
def task():
    print("Working")

task()
# Starting
# Working
# Done
```

Part 4: Handling Any Function with `*args` and `**kwargs`

Our wrappers so far only work on functions that take no arguments. Real functions take arguments, and a good decorator should work with *any* function regardless of its signature. The solution is to make the wrapper accept arbitrary arguments using `*args` and `**kwargs`, then pass them straight through to the original function.

A quick reminder on what those mean: `*args` collects any positional arguments into a tuple, and `**kwargs` collects any keyword arguments into a dictionary. Together they catch absolutely any combination of arguments.

python

```python
def debug(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args} {kwargs}")
        result = func(*args, **kwargs)       # forward everything along
        print(f"It returned {result}")
        return result                        # don't forget to return!
    return wrapper

@debug
def add(a, b):
    return a + b

print(add(2, 3))
# Calling add with (2, 3) {}
# It returned 5
# 5
```

Two habits to notice, because forgetting either is the most common beginner mistake:

- The wrapper **forwards** the arguments with `func(*args, **kwargs)`, so the original function still receives exactly what the caller intended.
- The wrapper **returns** the original function's result. If you forget the `return`, your decorated function will silently start returning `None`, which leads to baffling bugs.

Part 5: Preserving the Function's Identity with `functools.wraps`

There is a hidden cost to wrapping. When you replace a function with a wrapper, the wrapper's identity masks the original. The decorated function now reports the wrong name and loses its docstring:

python

```python
def logged(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@logged
def example():
    """I am the docstring."""
    pass

print(example.__name__)   # 'wrapper'  <- wrong!
print(example.__doc__)    # None        <- lost!
```

This breaks debugging tools, documentation generators, and anyone inspecting your code. The fix is built into the standard library: decorate your *wrapper* with `functools.wraps(func)`, which copies the original function's name, docstring, and other metadata onto the wrapper.

python

```python
import functools

def logged(func):
    @functools.wraps(func)          # copy func's identity onto wrapper
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@logged
def example():
    """I am the docstring."""
    pass

print(example.__name__)   # 'example'           <- correct!
print(example.__doc__)    # 'I am the docstring.' <- preserved!
```

Treat `@functools.wraps(func)` as a non-negotiable part of every decorator you write. It costs one line and saves real headaches.

Part 6: A Genuinely Useful Decorator — Timing

Let's put the pieces together into something you would actually reach for: a decorator that measures how long a function takes to run.

python

```python
import functools
import time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.6f} seconds")
        return result
    return wrapper

@timer
def crunch_numbers():
    return sum(range(1_000_000))

crunch_numbers()
# crunch_numbers took 0.013421 seconds
```

The beauty is that you can drop `@timer` on top of *any* function to instrument it, then remove the line when you are done — without ever editing the function's logic. This is the core selling point of decorators: behavior that can be cleanly added and removed, applied to many functions, all defined in one place.

Part 7: Decorators That Take Arguments

Sometimes you want to configure a decorator, like `@repeat(times=3)`. This requires one more layer of nesting, and it trips people up, so let's reason through it carefully.

When you write `@repeat(times=3)`, Python first *calls* `repeat(times=3)` and then uses whatever that returns as the actual decorator. So `repeat` is not a decorator itself — it is a **function that builds and returns a decorator**. That means we need three nested layers:

1. The outer function takes the *configuration* (`times`).
2. The middle function is the real decorator and takes the *function* (`func`).
3. The inner function is the wrapper and takes the *arguments* (`*args, **kwargs`).

```python
```

```python
import functools

def repeat(times):                          # 1. takes configuration
    def decorator(func):                    # 2. takes the function
        @functools.wraps(func)
        def wrapper(*args, **kwargs):       # 3. takes the call arguments
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def hello():
    print("hello")

hello()
# hello
# hello
# hello
```

The mental trick: `@repeat(times=3)` runs `repeat(times=3)` first, which hands back `decorator`, and *that* is what decorates `hello`. Three layers, each with one clear job: configure, wrap, execute.

Part 8: Stacking Decorators

You can apply more than one decorator to a single function by stacking them. They apply **from the bottom up** — the one closest to the function wraps first, then each one above wraps the result.

```python
```

```python
import functools

def bold(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return "<b>" + func(*args, **kwargs) + "</b>"
    return wrapper

def italic(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return "<i>" + func(*args, **kwargs) + "</i>"
    return wrapper

@bold
@italic
def text():
    return "hi"

print(text())     # <b><i>hi</i></b>
```

Read the stack from the function upward: `italic` wraps `text` first, giving `<i>hi</i>`, then `bold` wraps that, giving `<b><i>hi</i></b>`. The decorator nearest the function is applied first and therefore ends up on the inside.

Part 9: Class-Based Decorators

Functions are the most common way to write decorators, but a class can serve as one too. This is handy when the decorator needs to hold state across calls. A class becomes callable like a function when you define the `__call__` method. Here is a decorator that counts how many times a function has been called:

```python
```

```python
import functools

class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)   # the class-based equivalent of
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"Call #{self.count} of {self.func.__name__}")
        return self.func(*args, **kwargs)

@CountCalls
def hello():
    print("hi")

hello()   # Call #1 of hello \n hi
hello()   # Call #2 of hello \n hi
```

When you write `@CountCalls` above `hello`, Python passes `hello` to `CountCalls.__init__`, creating an instance that stores the function and a counter. Each call to `hello()` then triggers `__call__`, which bumps the counter and forwards to the real function. The state (`self.count`) lives naturally on the instance.

Part 10: Real-World Example Problems

Theory clicks when it solves real problems. Here are four decorators you might genuinely ship.

Problem 1: Caching Expensive Results (Memoization)

A naive recursive Fibonacci function recomputes the same values an astronomical number of times. A caching decorator stores results so each input is computed only once. The cache lives in the closure, shared across every call:

```python
```

```python
import functools

def memoize(func):
    cache = {}                          # lives in the closure
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)   # compute and store on first sight
        return cache[args]              # reuse thereafter
    return wrapper

@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50))   # 12586269025 — instant, instead of taking ages
```

This pattern is so useful that Python ships an optimized, battle-tested version in the standard library, `functools.lru_cache`. You should almost always prefer it over rolling your own:

```python
import functools

@functools.lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50))   # 12586269025
```

Problem 2: Retrying Flaky Operations

Network calls and other unreliable operations sometimes fail randomly. A retry decorator can automatically try again a few times before giving up — without cluttering your business logic with retry loops:

```python
```

```python
import functools
import time

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            attempts = 0
            while attempts < max_attempts:
                try:
                    return func(*args, **kwargs)
                except Exception as error:
                    attempts += 1
                    print(f"Attempt {attempts} failed: {error}")
                    if attempts >= max_attempts:
                        raise            # out of retries; re-raise the error
                    time.sleep(delay)
        return wrapper
    return decorator

@retry(max_attempts=3, delay=2)
def fetch_data():
    # imagine this calls an unreliable network service
    ...
```

Problem 3: Access Control / Authentication

Web applications constantly need to check whether a user is allowed to perform an action. A decorator keeps that check in one place and out of every individual function:

python

```python
import functools

def requires_role(role):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(user, *args, **kwargs):
            if user.get("role") != role:
                raise PermissionError(f"This action requires the '{role}' role")
            return func(user, *args, **kwargs)
        return wrapper
    return decorator

@requires_role("admin")
def delete_database(user):
    return "database deleted"

admin = {"name": "Sam", "role": "admin"}
guest = {"name": "Pat", "role": "guest"}

print(delete_database(admin))   # database deleted
print(delete_database(guest))   # raises PermissionError
```

This is essentially how decorators like Flask's `@login_required` work under the hood. The logic for "is this person allowed?" is written once and reused everywhere.

Problem 4: Logging Function Activity

In production systems you often want a record of which functions ran, with what inputs, and what they returned. A logging decorator gives you that consistently across an entire codebase:

```python
```

```
import functools
import logging

logging.basicConfig(level=logging.INFO)

def log_activity(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        logging.info(f"Running {func.__name__} with args={args} kwargs={kwargs}
        result = func(*args, **kwargs)
        logging.info(f"{func.__name__} returned {result}")
        return result
    return wrapper

@log_activity
def calculate_total(price, quantity):
    return price * quantity

calculate_total(9.99, 3)
```

Because the logging is centralized in the decorator, you can change the format, the log level, or the destination in one spot and every decorated function follows along.

Part 11: Common Pitfalls to Avoid

A handful of traps catch nearly everyone learning decorators:

Forgetting to return the result. If your wrapper calls `func(*args, **kwargs)` but does not `return` it, the decorated function silently returns `None`. This is the most frequent decorator bug. Always return the result unless you are certain the function returns nothing.

Forgetting `functools.wraps`. Without it, your decorated functions lose their names and docstrings, which breaks debugging and documentation tools. Make it a reflex on every wrapper.

Confusing `@deco` with `@deco()`. Use `@deco` when the decorator takes no configuration. Use `@deco()` (with parentheses) only when the decorator is the kind that takes arguments and must be *called* to produce the real decorator. Mixing these up produces confusing errors.

Calling the function instead of passing it. Remember that `my_decorator(say_hi)` passes the function, while `my_decorator(say_hi())` calls it first and passes the *result*. The missing or extra parentheses change everything.

Shared mutable state across calls. A cache or counter living in the closure is shared by every call to the decorated function. That is often exactly what you want (as with

memoization), but be deliberate about it, especially in multi-threaded code.

Summary

Here is the whole journey in a nutshell:

- In Python, **functions are objects**: they can be assigned to variables, passed as arguments, and returned from other functions.
- A function defined inside another remembers the outer variables it uses; this memory is a **closure**, and it powers decorators.
- A **decorator** is a function that takes a function, wraps it in extra behavior, and returns the wrapped version — leaving the original code untouched.
- `@decorator` above a definition is just shorthand for `func = decorator(func)`.
- A robust wrapper uses `*args, **kwargs` to accept any arguments, **returns** the original result, and uses `@functools.wraps(func)` to preserve the function's identity.
- **Decorators with arguments** need three nested layers: configure, wrap, execute.
- **Stacked** decorators apply from the bottom up, and **class-based** decorators (via `__call__`) are handy when you need to hold state.
- Real-world uses include caching, retrying, access control, logging, and timing — cross-cutting concerns written once and reused everywhere.

The practical rule of thumb: reach for a decorator whenever you find yourself wanting to add the *same* wrapping behavior — timing, logging, validation, caching, permissions — to many different functions. Instead of copying that behavior into each one, you write it once as a decorator and apply it with a single tidy `@` line. Recognizing those opportunities, and knowing exactly what the `@` is doing, is a real mark of a fluent Python programmer.